

```
1 # IMPORT LIBRARIES
2
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 from sklearn.model_selection import train_test_split
8 from sklearn.tree import DecisionTreeClassifier, plot_tree
9 from sklearn.preprocessing import OneHotEncoder
10 from sklearn.compose import ColumnTransformer
11 from sklearn.pipeline import Pipeline
12 from sklearn.metrics import (
13     accuracy_score,
14     precision_score,
15     recall_score,
16     f1_score,
17     classification_report,
18     confusion_matrix,
19     ConfusionMatrixDisplay
20 )
21
22 import warnings
23 warnings.filterwarnings("ignore", category=UserWarning)
```

```
1 # LOAD DATASET
2
3 from google.colab import files
4 uploaded = files.upload()
```

Lending Cl...ataset.xlsx

Lending Club dataset.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 765320 bytes, last modified: 9/2/2026 - 100% done
~~Saving Lending Club dataset.xlsx to Lending Club dataset (1).xlsx~~

```
1 # Read the uploaded Excel file
2
3 file_name = list(uploaded.keys())[0]
4 df = pd.read_excel(file_name)
```

```
1 # INITIAL DATA INSPECTION
2
3 print("First 5 rows:")
4 display(df.head())
```

First 5 rows:

	id	member_id	term	purpose	loan_status	loan_amnt	int_rate
0	62286683	66483442	36 months	credit_card	Fully Paid	24000	7.89
1	46314315	49422035	36 months	credit_card	Fully Paid	8000	6.68
2	51317198	54726945	36 months	credit_card	Fully Paid	12175	9.17

```
1 print("\nLast 5 rows:")
2 display(df.tail())
```

Last 5 rows:

	id	member_id	term	purpose	loan_status	loan_amnt	int_rate
7146	38332310	41116076	36 months	credit_card	Fully Paid	5000	17.1
7147	45374391	48482120	36 months	credit_card	Fully Paid	5000	8.1
7148	60771081	64812826	36 months	credit_card	Fully Paid	13800	8.1

```
1 print("\nDataset dimensions:")
2 print("Rows:", df.shape[0])
3 print("Columns:", df.shape[1])
```

Dataset dimensions:
Rows: 7151
Columns: 16

```
1 print("\nColumn names:")
2 print(df.columns.tolist())
```

Column names:
['id', 'member_id', 'term', 'purpose', 'loan_status', 'loan_amnt', 'int_rate']

```
1 # DATA TYPES AND INFORMATION
2
3 print("Dataset Information:")
4 df.info()
```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7151 entries, 0 to 7150
Data columns (total 16 columns):

```

#    Column      Non-Null Count  Dtype
---  -
0    id          7151 non-null    int64
1    member_id   7151 non-null    int64
2    term        7151 non-null    object
3    purpose     7151 non-null    object
4    loan_status 7151 non-null    object
5    loan_amnt   7151 non-null    int64
6    int_rate    7151 non-null    float64
7    installment 7151 non-null    float64
8    home_ownership 7151 non-null    object
9    annual_inc  7151 non-null    float64
10   verification_status 7151 non-null    object
11   revol_bal    7151 non-null    int64
12   revol_util   7148 non-null    float64
13   total_acc    7151 non-null    int64
14   acc_open_past_24mths 7151 non-null    int64
15   total_pymnt  7151 non-null    float64
dtypes: float64(5), int64(6), object(5)
memory usage: 894.0+ KB

```

```

1 # DESCRIPTIVE STATISTICS
2
3 print("Descriptive Statistics:")
4 display(df.describe())

```

Descriptive Statistics:

	id	member_id	loan_amnt	int_rate	installment	an
count	7.151000e+03	7.151000e+03	7151.000000	7151.000000	7151.000000	7.15
mean	5.106186e+07	5.453440e+07	13579.373514	10.431779	440.715538	7.62
std	9.419570e+06	1.003211e+07	8148.555935	3.316630	266.743864	5.45
min	5.800840e+05	7.457430e+05	1000.000000	5.320000	30.850000	8.00
25%	4.229390e+07	4.525578e+07	7500.000000	7.890000	243.130000	4.50
50%	5.044621e+07	5.381612e+07	11425.000000	9.990000	367.190000	6.50
75%	5.923142e+07	6.311782e+07	18600.000000	12.690000	609.360000	9.30
max	6.861588e+07	7.351869e+07	35000.000000	25.800000	1309.490000	1.90

```

1 # MISSING VALUE ANALYSIS
2
3 missing_values = df.isnull().sum()

```

```

1 missing_table = pd.DataFrame({
2     'Missing Values': missing_values,
3     'Percentage Missing': (missing_values / len(df)) * 100
4 })

```

```

1 display(missing_table)

```

	Missing Values	Percentage Missing	
id	0	0.000000	
member_id	0	0.000000	
term	0	0.000000	
purpose	0	0.000000	
loan_status	0	0.000000	
loan_amnt	0	0.000000	
int_rate	0	0.000000	
installment	0	0.000000	
home_ownership	0	0.000000	
annual_inc	0	0.000000	
verification_status	0	0.000000	
revol_bal	0	0.000000	
revol_util	3	0.041952	
total_acc	0	0.000000	
acc_open_past_24mths	0	0.000000	
total_pymnt	0	0.000000	

Next steps: [Generate code with missing_table](#) [New interactive sheet](#)

```
1 # DUPLICATE CHECK
2
3 print("Number of duplicate rows:", df.duplicated().sum())
```

Number of duplicate rows: 0

```
1 # TARGET VARIABLE ANALYSIS
2
3 print("Loan Status Counts:")
4 display(df['loan_status'].value_counts())
```

Loan Status Counts:

	count
Fully Paid	5943
Charged Off	1208

dtype: int64

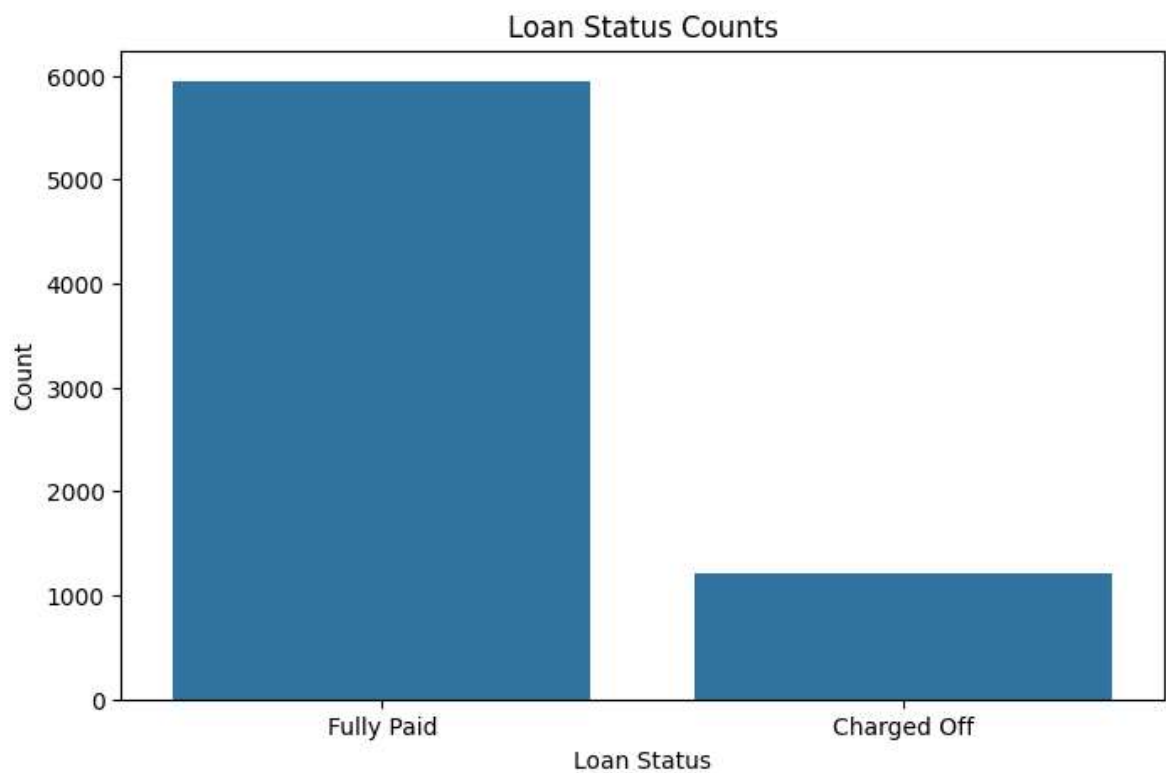
```
1 print("\nLoan Status Percentages:")
2 display(df['loan_status'].value_counts(normalize=True) * 100)
```

Loan Status Percentages:

	proportion
loan_status	
Fully Paid	83.107258
Charged Off	16.892742

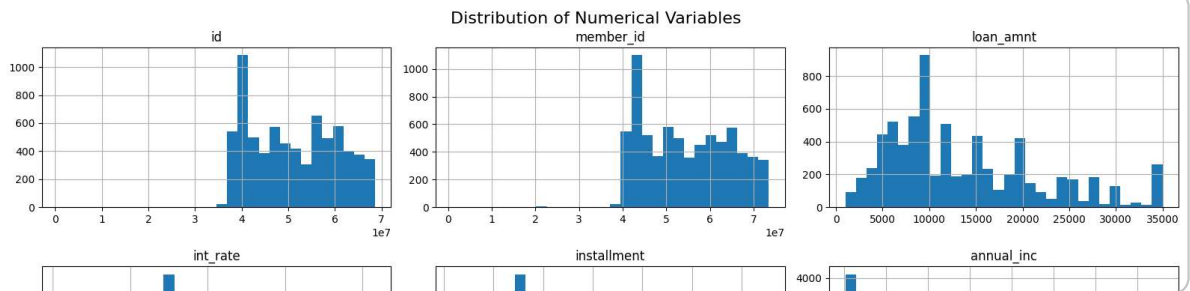
dtype: float64

```
1 # TARGET VARIABLE VISUALIZATION
2
3 plt.figure(figsize=(8, 5))
4 sns.countplot(data=df, x='loan_status')
5 plt.title('Loan Status Counts')
6 plt.xlabel('Loan Status')
7 plt.ylabel('Count')
8 plt.show()
```



```
1 # NUMERICAL VARIABLE DISTRIBUTIONS
2
3 numeric_columns = df.select_dtypes(include=np.number).columns
4 df[numeric_columns].hist()
```

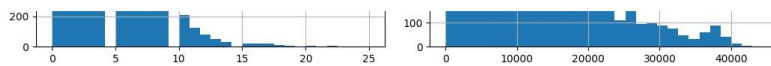
```
5     figsize=(16, 12),
6     bins=30
7 )
8 plt.suptitle(
9     'Distribution of Numerical Variables',
10    fontsize=16
11 )
12
13 plt.tight_layout()
14 plt.show()
```

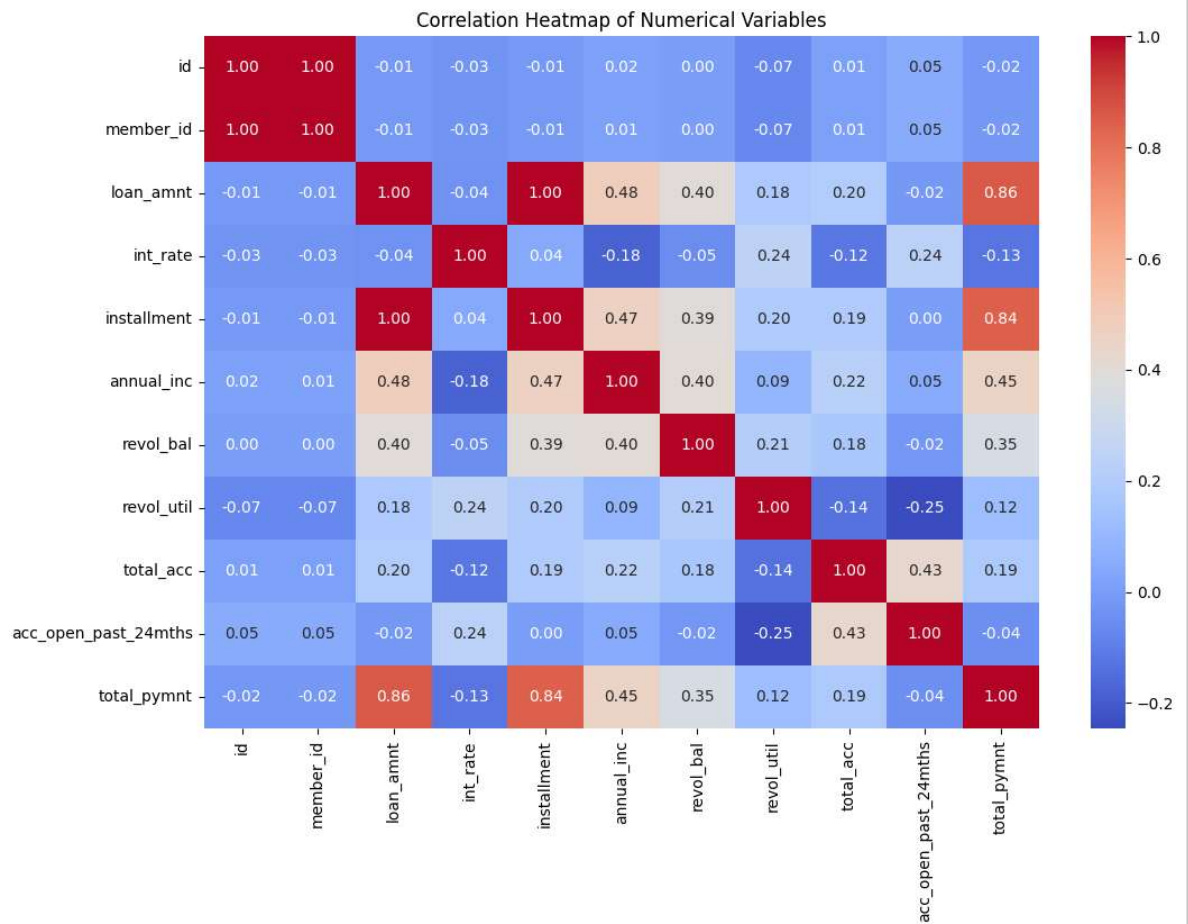



```

1 # CORRELATION HEATMAP
2 plt.figure(figsize=(12, 8))
3 correlation_matrix = df.select_dtypes(
4     include=np.number
5 ).corr()
6 sns.heatmap(
7     correlation_matrix,
8     annot=True,
9     fmt='.2f',
10    cmap='coolwarm'
11 )
12 plt.title('Correlation Heatmap of Numerical Variables')
13 plt.show()

```





```

1 # CATEGORICAL VARIABLE ANALYSIS
2
3 categorical_columns = df.select_dtypes(
4     include='object'
5 ).columns
6 for column in categorical_columns:
7     print("\n" + "="*20)
8     print(column)

```

```
9     print("="*20)
10    display(df[column].value_counts())
```



```
=====
term
=====
```

	count
term	

```
1 # LOAN STATUS VS CATEGORICAL VARIABLES
2 categorical_features = [
3     'home_ownership',
4     'verification_status'
5 ]
6 for column in categorical_features:
7     plt.figure(figsize=(9, 5))
8     sns.countplot(
9         data=df,
10        x=column,
11        hue='loan_status'
12    )
13     plt.title(f'Loan Status by {column}')
14     plt.xlabel(column)
15     plt.ylabel('Number of Loans')
16     plt.xticks(rotation=20)
17     plt.show()
```

```
count
```

loan_status	
Fully Paid	5943
Charged Off	1208

dtype: int64

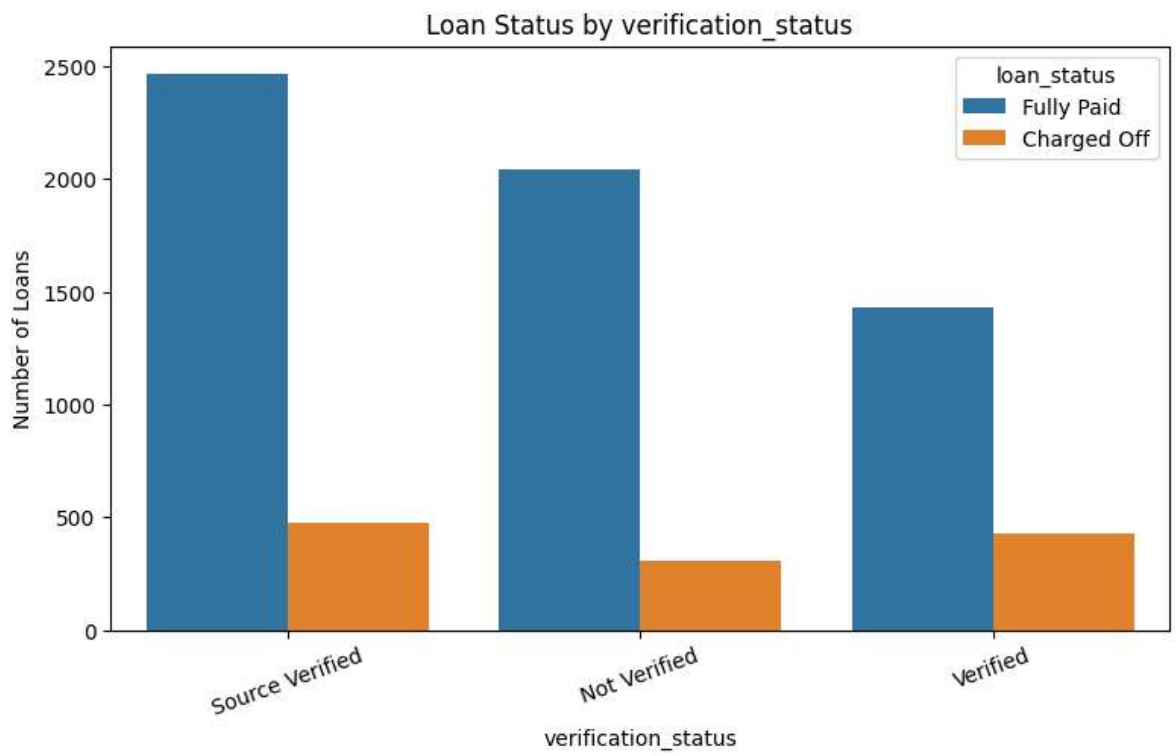
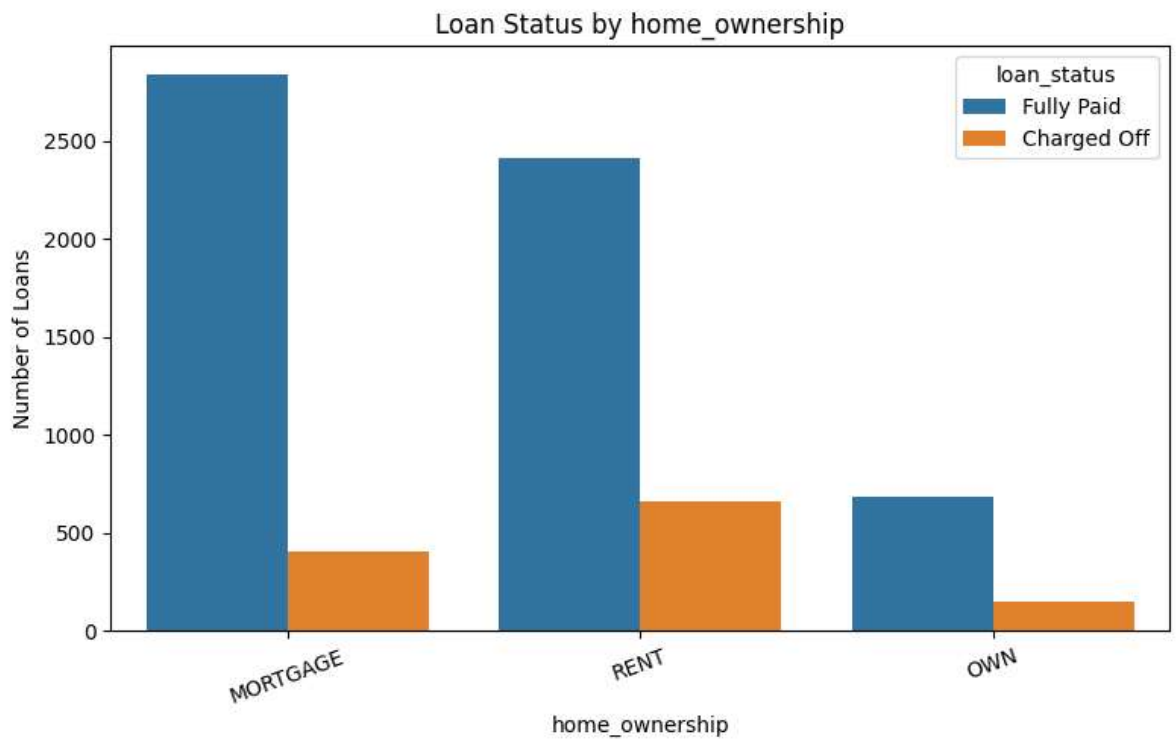
```
=====
home_ownership
=====
```

	count
home_ownership	
MORTGAGE	3243
RENT	3075
OWN	833

dtype: int64

```
=====
verification_status
=====
```

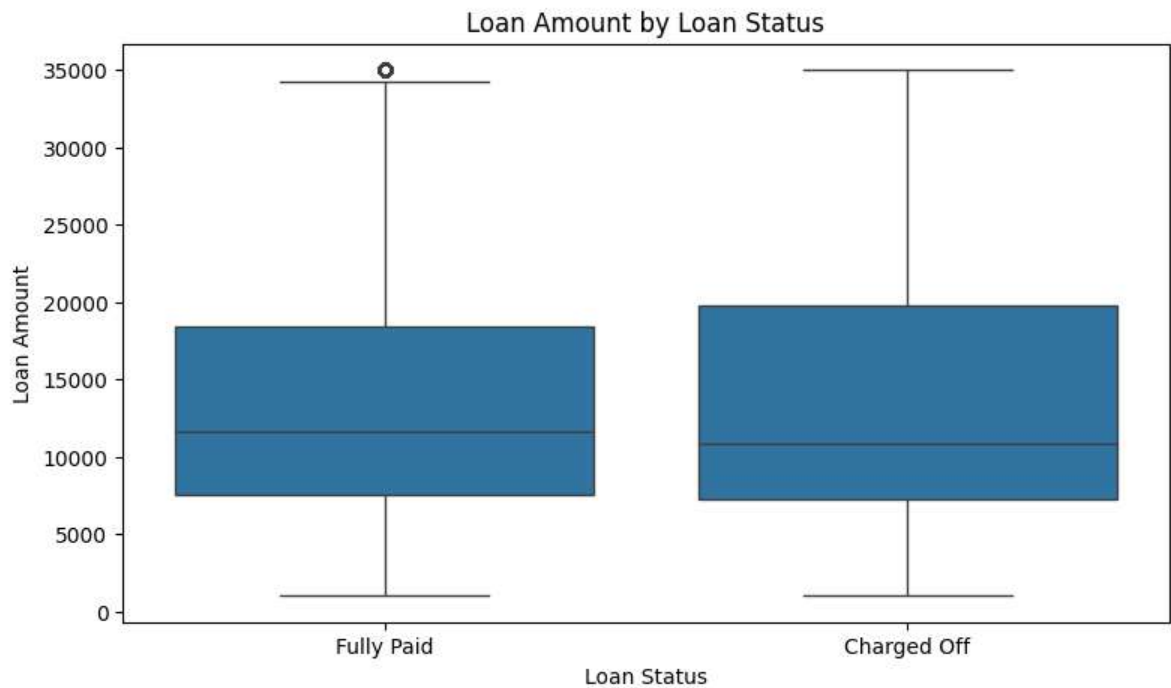
	count
verification_status	



```

1 # LOAN AMOUNT VS LOAN STATUS
2
3 plt.figure(figsize=(9, 5))
4 sns.boxplot(
5     data=df,
6     x='loan_status',
7     y='loan_amnt'
8 )
9 plt.title('Loan Amount by Loan Status')
10 plt.xlabel('Loan Status')
11 plt.ylabel('Loan Amount')
12 plt.show()

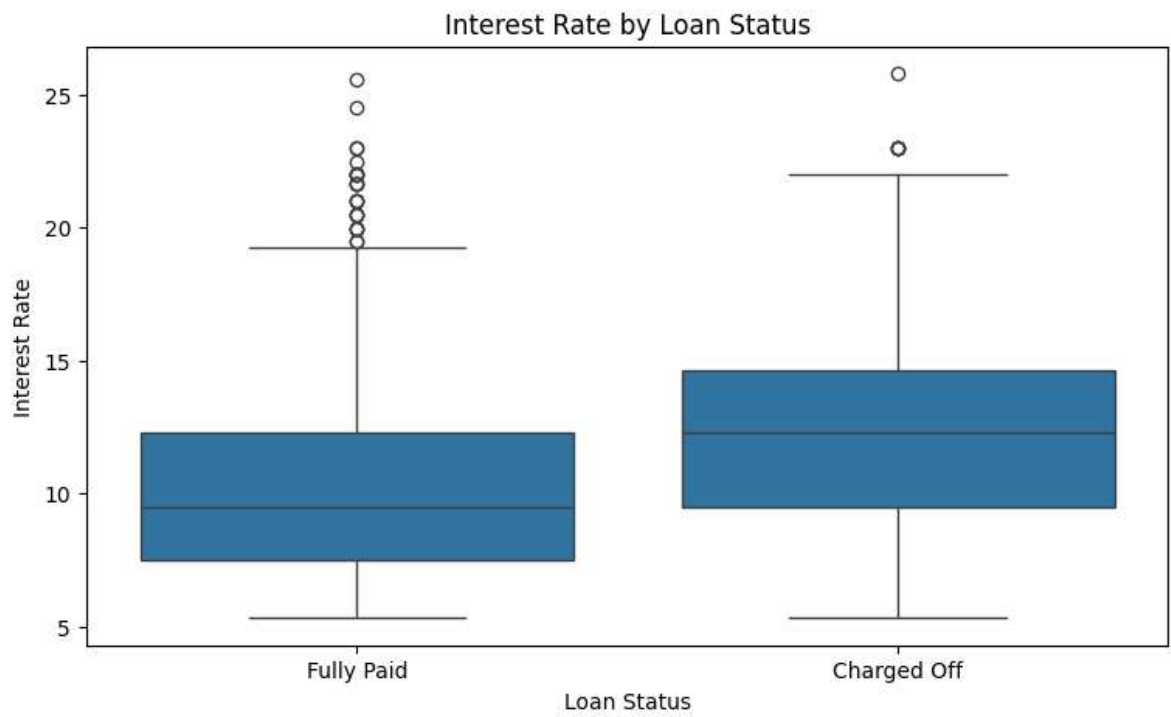
```



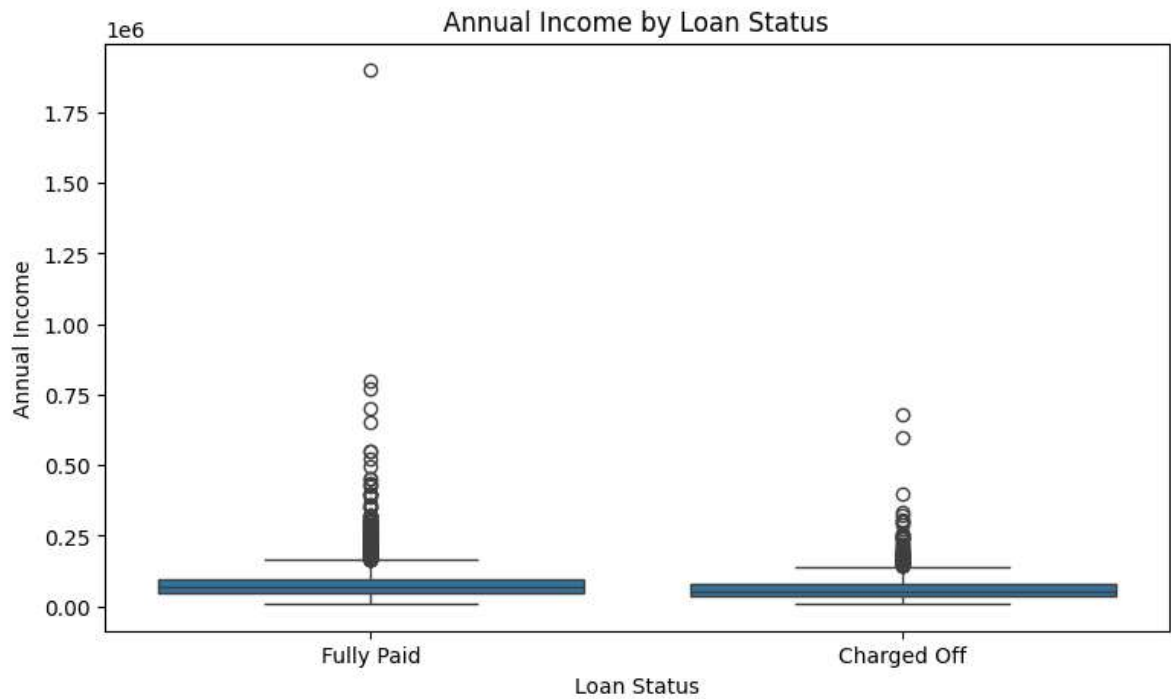
```

1 # INTEREST RATE VS LOAN STATUS
2
3 plt.figure(figsize=(9, 5))
4 sns.boxplot(
5     data=df,
6     x='loan_status',
7     y='int_rate'
8 )
9 plt.title('Interest Rate by Loan Status')
10 plt.xlabel('Loan Status')
11 plt.ylabel('Interest Rate')
12 plt.show()

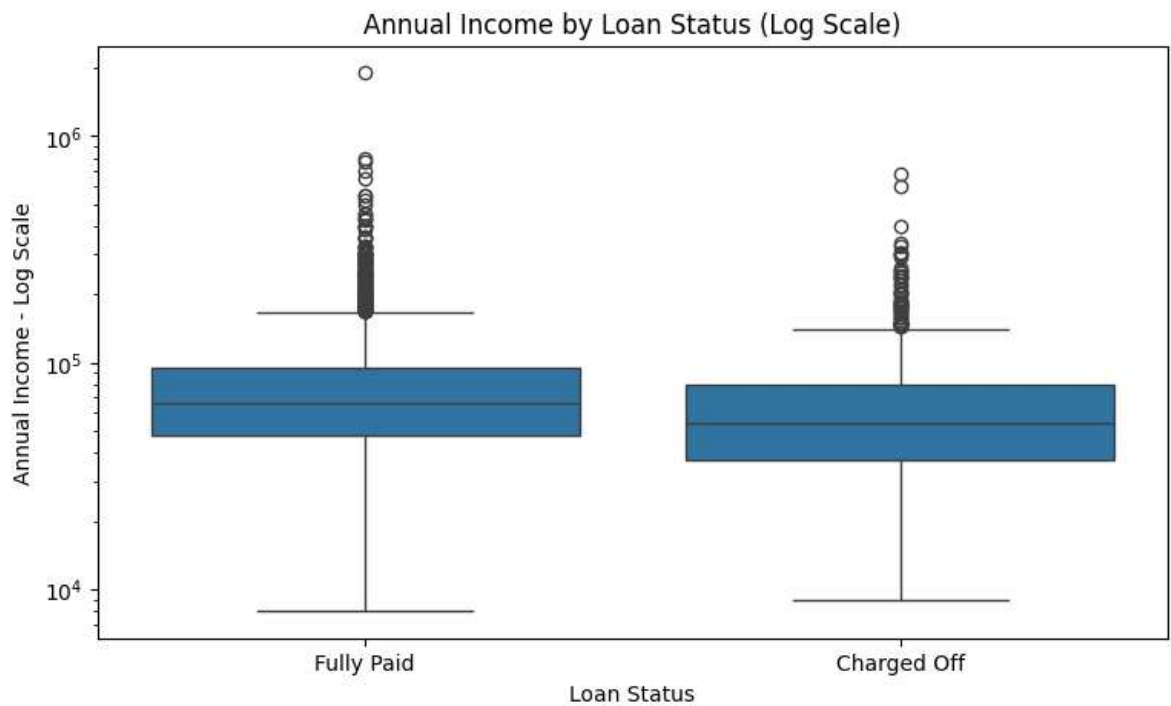
```



```
1 # ANNUAL INCOME VS LOAN STATUS
2
3 plt.figure(figsize=(9, 5))
4 sns.boxplot(
5     data=df,
6     x='loan_status',
7     y='annual_inc'
8 )
9 plt.title('Annual Income by Loan Status')
10 plt.xlabel('Loan Status')
11 plt.ylabel('Annual Income')
12 plt.show()
```



```
1 # Log-scale visualization of annual income
2
3 plt.figure(figsize=(9, 5))
4 sns.boxplot(
5     data=df,
6     x='loan_status',
7     y='annual_inc'
8 )
9 plt.yscale('log')
10 plt.title('Annual Income by Loan Status (Log Scale)')
11 plt.xlabel('Loan Status')
12 plt.ylabel('Annual Income - Log Scale')
13 plt.show()
```

```
1 # DEFINE FEATURES AND TARGET
2
3 features = [
4     'loan_amnt',
5     'int_rate',
6     'installment',
7     'home_ownership',
8     'annual_inc',
9     'verification_status',
10    'revol_bal',
11    'revol_util',
12    'total_acc',
13    'acc_open_past_24mths'
14 ]
15 X = df[features].copy()
16 y = df['loan_status'].copy()
```

```
1 print("Feature matrix shape:", X.shape)
```

Feature matrix shape: (7151, 10)

```
1 print("Target shape:", y.shape)
```

Target shape: (7151,)

```
1 print("\nFeatures used:")
2 print(features)
```

Features used:

```
['loan_amnt', 'int_rate', 'installment', 'home_ownership', 'annual_inc', 'ver
```

```
1 print("\nTarget classes:")
2 print(y.unique())
```

Target classes:

```
['Fully Paid' 'Charged Off']
```

```
1 # IDENTIFY FEATURE TYPES
2
3 numeric_features = [
4     'loan_amnt',
5     'int_rate',
6     'installment',
7     'annual_inc',
8     'revol_bal',
9     'revol_util',
10    'total_acc',
11    'acc_open_past_24mths'
12 ]
```

```
1 categorical_features = [
2     'home_ownership',
3     'verification_status'
4 ]
```

```
1 print("Numerical features:")
2 print(numeric_features)
```

Numerical features:

```
['loan_amnt', 'int_rate', 'installment', 'annual_inc', 'revol_bal', 'revol_ut
```

```
1 print("\nCategorical features:")
2 print(categorical_features)
```

Categorical features:

```
['home_ownership', 'verification_status']
```

```
1 # TRAIN-TEST SPLIT
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X,
5     y,
6     test_size=0.20,
```

```
7     random_state=42,  
8     stratify=y  
9 )
```

```
1 print("Training set size:", X_train.shape)  
2 print("Testing set size:", X_test.shape)
```

```
Training set size: (5720, 10)  
Testing set size: (1431, 10)
```

```
1 print("\nTraining target distribution:")  
2 print(y_train.value_counts())
```

```
Training target distribution:  
loan_status  
Fully Paid      4754  
Charged Off     966  
Name: count, dtype: int64
```

```
1 print("\nTesting target distribution:")  
2 print(y_test.value_counts())
```

```
Testing target distribution:  
loan_status  
Fully Paid      1189  
Charged Off     242  
Name: count, dtype: int64
```

```
1 # PREPROCESSING  
2  
3 from sklearn.impute import SimpleImputer  
4  
5 # Numerical preprocessing:  
6 # Missing values are replaced with the median  
7 numeric_transformer = Pipeline(  
8     steps=[  
9         ('imputer', SimpleImputer(strategy='median'))  
10    ]  
11 )
```

```
1 # Categorical preprocessing:  
2 # Missing values are replaced with the most frequent category  
3 # and categorical variables are converted to dummy variables  
4 categorical_transformer = Pipeline(  
5     steps=[  
6         ('imputer', SimpleImputer(strategy='most_frequent')),  
7         ('onehot', OneHotEncoder(handle_unknown='ignore'))  
8     ]  
9 )
```

```
1 # Combine both preprocessing methods
2 preprocessor = ColumnTransformer(
3     transformers=[
4         ('num', numeric_transformer, numeric_features),
5         ('cat', categorical_transformer, categorical_features)
6     ]
7 )
```

```
1 print("Preprocessing pipeline created successfully.")
```

Preprocessing pipeline created successfully.

```
1 # CART MODEL
2
3 cart_model = Pipeline(
4     steps=[
5         ('preprocessor', preprocessor),
6         ('classifier', DecisionTreeClassifier(
7             criterion='gini',
8             random_state=42,
9             max_depth=5
10        ))
11     ]
12 )
13 print("CART model created successfully.")
```

CART model created successfully.

```
1 # TRAIN CART MODEL
2
3 cart_model.fit(X_train, y_train)
4 print("CART model trained successfully.")
```

CART model trained successfully.

```
1 # MAKE PREDICTIONS
2
3 y_pred = cart_model.predict(X_test)
4 print("Predictions generated successfully.")
5 print("\nFirst 20 predictions:")
6 print(y_pred[:20])
```

Predictions generated successfully.

First 20 predictions:

```
['Fully Paid' 'Fully Paid' 'Fully Paid' 'Fully Paid' 'Fully Paid'
 'Fully Paid' 'Fully Paid' 'Fully Paid' 'Fully Paid' 'Fully Paid'
 'Fully Paid' 'Fully Paid' 'Fully Paid' 'Fully Paid' 'Fully Paid'
 'Fully Paid' 'Fully Paid' 'Fully Paid' 'Fully Paid' 'Fully Paid']
```

```
1 # MODEL ACCURACY
2
3 accuracy = accuracy_score(y_test, y_pred)
```

```

4 print("CART Model Accuracy:")
5 print(f"{accuracy:.4f}")
6 print(f"\nAccuracy: {accuracy * 100:.2f}%")

```

CART Model Accuracy:
0.8274

Accuracy: 82.74%

```

1 # CLASSIFICATION METRICS
2
3 precision = precision_score(
4     y_test,
5     y_pred,
6     pos_label='Charged Off'
7 )
8 recall = recall_score(
9     y_test,
10    y_pred,
11    pos_label='Charged Off'
12 )
13 f1 = f1_score(
14     y_test,
15     y_pred,
16     pos_label='Charged Off'
17 )

```

```

1 print(f"Accuracy : {accuracy:.4f}")
2 print(f"Precision: {precision:.4f}")
3 print(f"Recall   : {recall:.4f}")
4 print(f"F1 Score : {f1:.4f}")

```

Accuracy : 0.8274
Precision: 0.4194
Recall : 0.0537
F1 Score : 0.0952

```

1 # CLASSIFICATION REPORT
2
3 print("Classification Report:")
4 print(
5     classification_report(
6         y_test,
7         y_pred
8     )
9 )

```

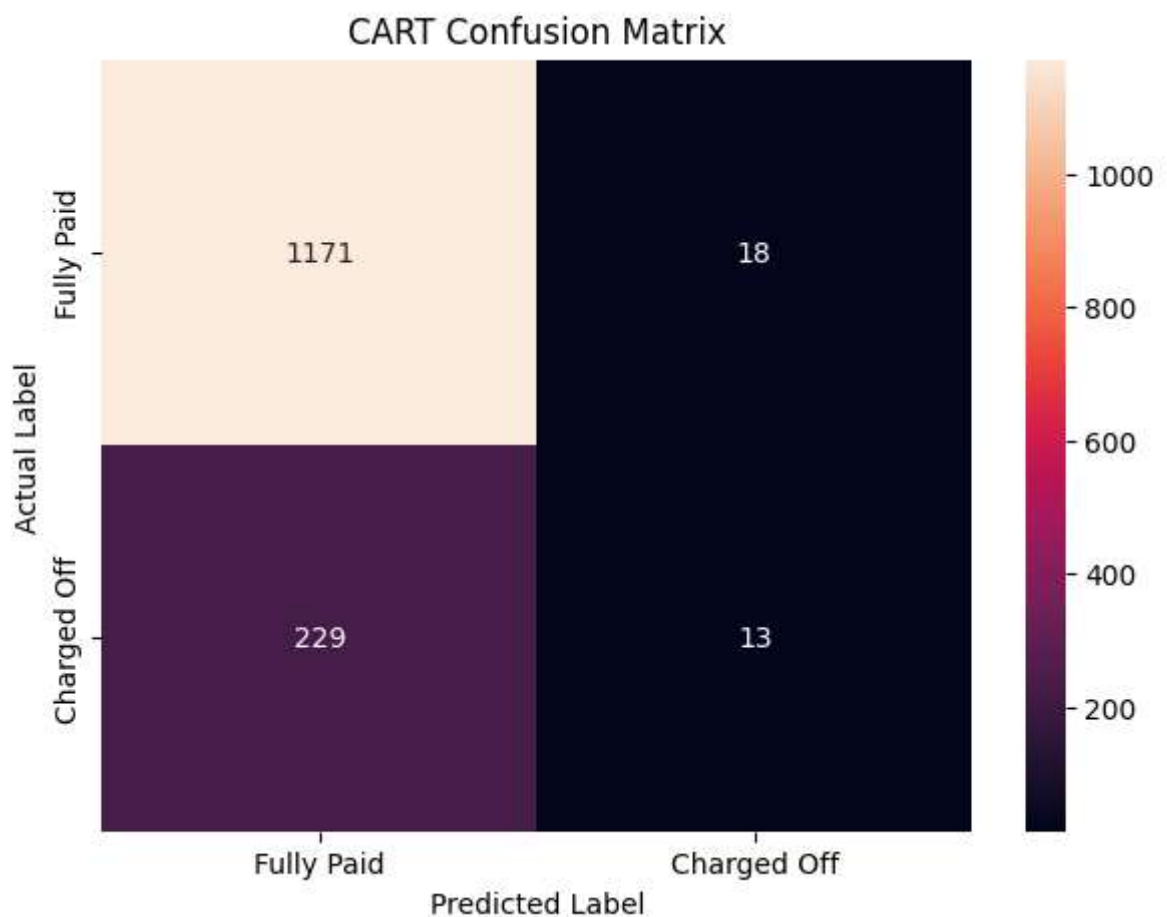
Classification Report:	precision	recall	f1-score	support
Charged Off	0.42	0.05	0.10	242
Fully Paid	0.84	0.98	0.90	1189
accuracy			0.83	1431

macro avg	0.63	0.52	0.50	1431
weighted avg	0.77	0.83	0.77	1431

```

1 # CONFUSION MATRIX
2
3 cm = confusion_matrix(
4     y_test,
5     y_pred,
6     labels=['Fully Paid', 'Charged Off']
7 )
8 plt.figure(figsize=(7, 5))
9 sns.heatmap(
10     cm,
11     annot=True,
12     fmt='d',
13     xticklabels=['Fully Paid', 'Charged Off'],
14     yticklabels=['Fully Paid', 'Charged Off']
15 )
16 plt.title('CART Confusion Matrix')
17 plt.xlabel('Predicted Label')
18 plt.ylabel('Actual Label')
19 plt.show()

```



```

1 # VISUALIZE CART DECISION TREE
2
3 # Get the trained decision tree
4 tree_model = cart_model.named_steps['classifier']

```

```
5
6 # Get feature names after one-hot encoding
7 feature_names = cart_model.named_steps[
8     'preprocessor'
9 ].get_feature_names_out()
10
11 plt.figure(figsize=(24, 12))
12 plot_tree(
13     tree_model,
14     feature_names=feature_names,
15     class_names=tree_model.classes_,
16     filled=True,
17     rounded=True,
18     fontsize=9
19 )
20 plt.title('CART Decision Tree')
21 plt.show()
```



```
3 importance = tree_model.feature_importances_  
4 feature_importance_df = pd.DataFrame({  
5     'Feature': feature_names,  
6     'Importance': importance  
7 })  
8 feature_importance_df = feature_importance_df.sort_values(  
9     by='Importance',  
10    ascending=False  
11 )  
12 display(feature_importance_df)
```